Which Wrong Pages Should an Event Linker Train On?

Negative sampling for the retrieval stage of Event Linking

Rodrigo Quezada
CS 2242, Fall 2026

First draft, November 6, 2026

Abstract

Event linking connects a mention of an event in text to the Wikipedia page about that event. The retrieval stage of the EVELINK system is trained with a contrastive loss against wrong pages, called negatives. This paper asks which wrong pages the retriever should train on. I compare four samplers at a fixed budget: the original in-batch negatives, uniform negatives from the top 1,000 retrieved pages, the seven highest-scoring pages, and ReSa2, a two-stage sampler that prefers pages that score close to the correct page and are similar to it. On 16,692 validation mentions ranked against 5.87 million pages, ReSa2 gives recall at ten of 93.3% against 92.2% for uniform sampling, 87.2% for top-score sampling, and 69.6% for in-batch sampling, with the correct page first 69.5% of the time against 64.3% for uniform. I also measure the inclusion probability, coverage, and variation between runs of each sampler, and I report a training collapse that appears with hard negatives at a higher learning rate and does not recover.

1 Introduction

EVELINK is a system that reads a mention of an event and tries to find the Wikipedia page for that event. Yu and colleagues introduced it in 2023 [1]. You give it a phrase like "won the Rising Star award," along with the sentence around it. EVELINK uses that context to work out which award the writer is talking about.

Take this sentence. "He won the Rising Star award in 1996, in his first season with Adelaide."

The page we want is "1996 AFL Rising Star." A page about the 2004 Indian Ocean earthquake is easy to rule out. The page for the 1998 AFL Rising Star is harder. It is the same award. The year is what tells us which page to link to. The Event Linking paper calls this a time error; its other common error, a hierarchy error, confuses an event with a parent event or a sub-event. In both cases the wrong page shares much of its wording with the correct page.

EVELINK makes that decision in two stages. First, a retriever searches Wikipedia and pulls out 100 candidate pages. Then a reranker reads the mention alongside each candidate and puts the pages in a new order. If the retriever leaves out the 1996 page, the reranker cannot bring it back.

That is where my project starts. During training, the retriever sees the correct page alongside a few wrong pages. I wanted to know which wrong pages would help it learn to put the right one higher. Two questions guide the paper:

1. How does ReSa2 [2], a two-stage sampler of wrong pages, change the inclusion probability of each candidate, the coverage of the candidate pool, and the variation between runs?
2. With a fixed compute budget, does ReSa2 put the correct page higher (recall at ten) than the original sampling?

2 The math behind the retriever

The retriever uses two BERT encoders. One reads the mention and its context. The other reads the Wikipedia page. Each encoder turns its input into a vector with 1,024 numbers. I call the mention vector u_m and the page vector v_e.

To score a page, I take the dot product. I multiply the first number in one vector by the first number in the other, do the same for the rest, and add up the products:

$$s(m, e) = u_m \cdot v_e = \sum_{j=1}^{1024} u_{m,j}\, v_{e,j}. \tag{1}$$

The retriever puts pages with higher scores first. The two encoders work separately, so I can compute the page vectors ahead of time. When a new mention comes in, I encode it once and compare that vector with the stored page vectors.

During training, I give the retriever the correct page and seven wrong pages. I call the wrong pages *negatives*. If the correct page scores s^+ and the negatives score s_1 through s_7, the loss is

$$L = -\log \frac{e^{s^+}}{e^{s^+} + \sum_{i=1}^{7} e^{s_i}}. \tag{2}$$

The fraction is the correct page's share of the total after I exponentiate the eight scores. When the correct page gets almost all of that share, the loss is close to zero. When a wrong page scores about as high as the correct one, it takes a bigger share and the loss goes up.

To see the difference, suppose the correct page scores 10 and there is just one negative. If that negative scores -10, the loss is about two billionths. If it scores 9.5, the loss is about 0.47. The correct page has the same score in both cases. All I changed was the wrong page.

The math puts a bound on this. Write $g_i = s^+ - s_i$ for the gap by which negative i scores below the correct page. Then

$$L = \log\Big(1 + \sum_{i=1}^{7} e^{-g_i}\Big) \le \sum_{i=1}^{7} e^{-g_i}, \tag{3}$$

because $\log(1 + x) \le x$. A negative g points below the correct page adds at most e^{-g} to the loss. At $g = 20$ that is about 2×10^{-9}; at $g = 0.5$ it is about 0.61.

I can also ask how much the loss changes if I raise one negative's score. The derivative is that negative's share of the total:

$$p_i = \frac{\partial L}{\partial s_i} = \frac{e^{s_i}}{e^{s^+} + \sum_{j=1}^{7} e^{s_j}}. \tag{4}$$

So if the retriever already puts the earthquake page far below the 1996 award, that page contributes almost nothing to the update. If the 1998 award scores close to the 1996 award, the loss puts more pressure on the model to separate them.

The update itself moves the mention vector. By the chain rule through equation 1,

$$\nabla_{u_m} L = \sum_{i=1}^{7} p_i \, (v_{e_i} - v_{e^+}), \tag{5}$$

a weighted sum of the differences between each negative's page vector and the correct page vector, with the weights p_i from equation 4. A negative that scores far below the correct page has $p_i \approx 0$ and moves nothing. A negative that scores close to it has a large p_i, and the step is along $v_{e_i} - v_{e^+}$; if the two pages are near-duplicates, that difference is small, which matters in section 7.

In the experiment, I keep the loss and the number of negatives fixed. I change which seven pages the model sees.

3 How I chose the seven pages

I compared four ways of picking negatives. The first uses the correct pages for the other seven mentions in the same small batch. That is the in-batch baseline. For the other three, I started with the pages that the released retriever ranked highest for that mention. I built a pool of up to 1,000 candidates and excluded the correct page from sampling. Every sampler sees the same input: the pool scores s_i, the score s^+ of the correct page, and the similarity $\sigma_i = v_{e_i} \cdot v_{e^+}$ of each pool page to the correct page.

Uniform sampling gives every page in the pool the same chance, $k/N = 7/1{,}000$. Top-score sampling always takes the seven highest-scoring pages, so its inclusion probability is 1 for those seven pages and 0 for the rest. ReSa2 makes the choice in two stages.

First, ReSa2 draws 500 pages without replacement. It gives the most weight to pages that score close to the correct page:

$$w_i = \exp\!\big(-\tau \, (s_i - s^+)^2\big), \qquad \tau = 0.25. \tag{6}$$

It draws one page with probability $w_i / \sum_j w_j$, removes it, and draws again from the rest (successive sampling). The weight drops as the score moves away in either direction. Then it compares the 500 survivors to the correct page itself. It keeps the 75 pages whose vectors are most similar to the correct page vector and draws seven of those uniformly.

That last draw gives me a simple bound. Once a page makes the final 75, it has a 7-in-75 chance

of being used. So even if a page labeled as wrong is actually a valid link, I can bound how often ReSa2 trains against it and, with equation 4, bound its expected effect on the loss gradient:

$$\mathbb{E}[p_i] \le P(\text{ReSa2 picks } i) \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}} \le \frac{7}{75} \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}}. \tag{7}$$

3.1 What is proved in Lean

Three claims from this section are formalized in Lean 4 (core library only, no Mathlib; the file is in Appendix A). A page is modeled by its score, an integer, and a *possible false negative* is a page that scores above the correct page, $s_i > s^+$.

1. **A bound for every sampler.** A selection of k pages taken from a pool that contains A pages above s^+ holds at most $\min(k, A)$ of them. (`any_sampler_le`)
2. **Top-score reaches the bound.** The top-score rule selects exactly $\min(k, A)$ pages above s^+. (`topscore_is_max`) Together, 1 and 2 say that no sampler trains against more possible false negatives than top-score, for every pool and every run; the comparison with ReSa2 needs no assumption about ReSa2.
3. **The 7/75 share.** Of all 7-page subsets of 75 pages, the share that contains one fixed page is 7/75, that is, $75\binom{74}{6} = 7\binom{75}{7}$. (`stage2_share`, checked by computation) So $P(\text{ReSa2 selects } i) = \frac{7}{75} P(i \text{ is in the top 75})$.

The Lean statements are short enough to compare with the claims above:

```
theorem any_sampler_le (sPos : Int) (pool sel : List Int) (h : sel.Sublist pool) :
    nAbove sPos sel ≤ min sel.length (nAbove sPos pool)

theorem topscore_is_max (sPos : Int) (k : Nat) (pool : List Int) :
    nAbove sPos (topscore sPos k pool) = min k (nAbove sPos pool)

theorem stage2_share : 75 * choose 74 6 = 7 * choose 75 7
```

What is not formalized: the e^{-g} bound of equation 3 and the derivative of equation 4 need real numbers, exp and log, which core Lean does not provide; they are checked by hand above. The inclusion probability of stage 1 (successive sampling) is checked numerically: the exact enumeration for pools of up to 8 pages matches a simulation of the sampler to within 0.015. The training results of sections 5 and 6 are empirical.

4 Experiment setup

I started each run from the released EVELINK model. I froze the page encoder and trained the mention encoder for 1,500 updates with a batch of 32 mentions (four micro-batches of 8), AdamW, a learning rate of 10^{-5} with 10% warm-up, gradient clipping at 1.0, and seven negatives per mention. Because the page encoder is frozen, the pool scores and similarities are fixed before training. I ran

Table 1: Validation recall after 1,500 updates, mean ± standard deviation over three seeds. 16,692 mentions ranked against 5.87 million pages.

Sampler	Top-1	Recall@10	Recall@100
Released model, no fine-tuning	42.6	77.7	93.1
In-batch (EVELINK)	33.6 ± 1.9	69.6 ± 1.8	89.8 ± 1.3
Uniform from the pool	64.3 ± 0.2	92.2 ± 0.1	98.4 ± 0.1
Top-score	65.3 ± 1.2	87.2 ± 0.9	95.9 ± 0.6
ReSa2	**69.5 ± 0.3**	**93.3 ± 0.1**	**98.5 ± 0.1**

each of the four samplers with three seeds, twelve runs in total. Each run took about 56 minutes on one NVIDIA A10G.

Then I ranked all 5.87 million pages for each of the 16,692 validation mentions. The metric I focus on is recall at ten: if the correct page appears anywhere in the first ten results, I count that mention as a hit,

$$\text{Recall@}K = \frac{1}{|V|} \sum_{m \in V} \mathbf{1}[\text{rank}(e_m^*) \le K] \,. \tag{8}$$

I also report top-1 and recall at 100.

Before the experiments I reproduced the released pipeline: the released retriever and reranker give 82.88% accuracy on the Wikipedia test set (paper: 82.87%) and 55.33% on the New York Times set (paper: 55.33%). The test set is untouched by everything below.

5 What changed in the rankings

Table 1 gives the results. The released retriever got 77.7 percent recall at ten. After fine-tuning, in-batch sampling got 69.6. Uniform sampling from the pool got 92.2. Taking the seven highest-scoring negatives got 87.2. ReSa2 got 93.3.

Most of the improvement came from drawing negatives from the retrieved pool. ReSa2 added 1.1 percentage points over uniform sampling at ten. The difference was larger at rank one. With uniform sampling, the correct page came first 64.3 percent of the time. With ReSa2, it came first 69.5 percent of the time. The spread between seeds is 0.1 to 0.3 points for both samplers, so the gap is about ten times the noise.

In-batch sampling finished below the released model. At this budget that is expected: seven random negatives per step give almost no gradient (equation 3), while the original training used many more negatives per step and ran for longer.

6 Which pages the samplers pick

I also wanted to see which pages the samplers actually picked. I drew negatives 200 times each for 500 validation mentions. Table 2 gives the result.

Table 2: What each sampler selects: 500 validation mentions, 200 draws per mention, seven negatives per draw. The first column splits the seven negatives by the rank of their similarity to the correct page.

Sampler	Negatives by similarity rank top 25 / 25–75 / 75–300 / rest	Distinct pages (of 1,000)	Overlap of two draws	Score gap to correct page
Uniform	0.17 / 0.34 / 1.55 / 4.95	756	0.00	−13.3
Top-score	4.13 / 1.07 / 1.17 / 0.62	7	1.00	−1.4
ReSa2	2.15 / 3.98 / 0.87 / 0.00	84	0.05	−8.6

ReSa2 picked about six of its seven negatives from the 75 pages most similar to the correct page. For uniform sampling, the average was about half a page. Top-score gave me the same seven pages every time. Two ReSa2 draws shared very few pages (Jaccard overlap 0.05), even though both tended to draw from the same part of the pool. Over 200 draws uniform sampling touched 756 pool pages, ReSa2 84, top-score 7.

The score gap says how hard the negatives are. ReSa2's negatives score 8.6 points below the correct page on average, between uniform (−13.3, easy) and top-score (−1.4, hardest). With raw scores of about 550 and differences of tens of points, the weight in equation 6 is near zero outside $|s_i - s^+| < 4$, so in practice stage 1 is a window around s^+, and stage 2 picks the most similar pages inside that window.

7 What happened at the higher learning rate

At a learning rate of 3×10^{-5}, one ReSa2 run and one top-score run collapsed. The training loss jumped once, near the end of the warm-up, from about 0.13 to about 0.3, and stayed there. When I looked at the scores, I found that the mention encoder gave almost every page a score of about two. It gave roughly the same score to the correct page and to a random page; a healthy model gives about 550 to the correct page and 545 to a random one. The weights stayed finite. The loss over the eight training pages could still look modest while the ranking across millions of pages had fallen apart.

One possible explanation is in the gradient, equation 5. When a negative scores close to the correct page, it gets a substantial share of the training signal. If the two page vectors are also similar, the update depends on the small difference between them. My hypothesis is that repeated updates at the higher learning rate weakened the part of the mention vector that lined up with the page vectors. The scores are consistent with that explanation. Uniform and in-batch runs, whose negatives are far from the correct page, never jumped.

I tried training the collapsed model for another 1,500 updates. I tried ReSa2 negatives in one run and uniform negatives in another. Recall at ten stayed at zero in both. When I restarted from the released checkpoint and used a learning rate of 10^{-5}, all twelve runs trained without collapsing, and every sampler scored better than at the higher rate.

8 Conclusion

I started with the difference between the 1996 award and the 1998 award. The results make that example more specific for me. I want the retriever to see pages that are close enough to make it pay attention to the year. I also want it to see a different set of pages the next time it trains on that mention. In these runs, ReSa2 gave me that variety and put the correct page first more often than any of the other samplers I tested.

A The Lean file

`lean/FalseNegatives.lean` in the project repository, Lean 4 v4.34.1, core library only. It builds with `lake build`.

```
/-!
# Small Lean model of the false-negative claim in the proposal

Core Lean 4 only (no Mathlib). Check with:  cd lean && lake build

Setup. A page is its score (an integer, to stay simple). The correct page has
score `sPos`. A "possible false negative" is a page that scores above `sPos`.
A sampler returns a list of pages taken from the pool.

Main result (`topscore_is_max`): the top-score rule selects `min k A` possible
false negatives, where `A` is how many the pool has, and NO sampler of `k`
pages can select more (`any_sampler_le`). So "ReSa2 selects fewer of them than
top-score, in expectation" is true for every sampler, every pool, and every
run. It needs no condition on ReSa2.
-/

/-- `s` scores above the correct page. -/
def above (sPos s : Int) : Bool := decide (sPos < s)

/-- Number of possible false negatives in a list of pages. -/
def nAbove (sPos : Int) (l : List Int) : Nat := l.countP (above sPos)

/-! ## 1. An upper bound for every sampler -/

/-- Any selection taken from the pool (each page used at most once) has at most
`min (its size) A` possible false negatives. (Write the selected pages in
pool order; then "each page used at most once" means `sel` is a sublist.) -/
theorem any_sampler_le (sPos : Int) (pool sel : List Int) (h : sel.Sublist pool) :
    nAbove sPos sel ≤ min sel.length (nAbove sPos pool) :=
  Nat.le_min.mpr ⟨List.countP_le_length, h.countP_le⟩
```

```
/-! ## 2. The top-score rule reaches the bound

Sorting by score puts every page above ‘sPos‘ before every page below it.
So, for counting, top-score is: pages above first, then the others, take ‘k‘. -/

def topscore (sPos : Int) (k : Nat) (pool : List Int) : List Int :=
  (pool.filter (above sPos) ++ pool.filter (fun s => !above sPos s)).take k

/-- Taking ‘k‘ from (all-good list ++ no-good list) gives ‘min k (good length)‘ goods.
    -/
theorem countP_take_append (p : Int → Bool) (a b : List Int)
    (ha : ∀ x ∈ a, p x = true) (hb : ∀ x ∈ b, p x = false) :
    ∀ k, ((a ++ b).take k).countP p = min k a.length := by
  induction a with
  | nil =>
    intro k
    simp only [List.nil_append, List.length_nil, Nat.min_zero]
    exact List.countP_eq_zero.mpr fun x hx => by
      simp [hb x (List.mem_of_mem_take hx)]
  | cons x xs ih =>
    intro k
    cases k with
    | zero => simp
    | succ k =>
      have hx : p x = true := ha x (by simp)
      have ih' := ih (fun y hy => ha y (by simp [hy])) k
      simp only [List.cons_append, List.take_succ_cons, List.countP_cons, hx,
        ite_true, List.length_cons, ih']
      omega

theorem topscore_is_max (sPos : Int) (k : Nat) (pool : List Int) :
    nAbove sPos (topscore sPos k pool) = min k (nAbove sPos pool) := by
  unfold nAbove topscore
  rw [countP_take_append (above sPos) _ _ (fun x hx => (List.mem_filter.mp hx).2)
      (fun x hx => by simpa using (List.mem_filter.mp hx).2)]
  rw [List.countP_eq_length_filter]

/-! ## 3. Stage 2 of ReSa2 picks 7 of the top 75 uniformly

Counting fact: of all 7-page subsets of 75 pages, the share that contains one
fixed page is 7/75. So P(page selected) = (7/75) · P(page is in the top 75),
which is at most (7/75) · P(page is kept by stage 1). -/

/-- Binomial coefficient. Step i: C(n, i+1) = C(n, i) · (n − i) / (i + 1),
```

```
    and each division is exact. -/
def choose (n k : Nat) : Nat :=
  (List.range k).foldl (fun c i => c * (n - i) / (i + 1)) 1

/-- Subsets that contain the fixed page: choose the other 6 from 74.
    `75 · C(74,6) = 7 · C(75,7)`, that is, the share is 7/75. -/
theorem stage2_share : 75 * choose 74 6 = 7 * choose 75 7 := by decide

/-! ## Example with numbers -/

-- Pool: correct page at 0; three pages above it.
#eval nAbove 0 [5, -3, 1, -8, 2, -1]                 -- 3
#eval topscore 0 2 [5, -3, 1, -8, 2, -1]             -- [5, 1]
#eval nAbove 0 (topscore 0 7 [5, -3, 1, -8, 2, -1])  -- 3 = min 7 3
```

References

[1] Xiaodong Yu, Wenpeng Yin, Nitish Gupta, and Dan Roth. Event Linking: Grounding Event Mentions to Wikipedia. In *Proceedings of EACL*, 2023.

[2] Li et al. ReSa2: Re-sampling for Retrieval Training. In *Proceedings of ACML*, 2025.

[3] Ledell Wu, Fabio Petroni, Martin Josifoski, Sebastian Riedel, and Luke Zettlemoyer. Scalable Zero-shot Entity Linking with Dense Entity Retrieval. In *Proceedings of EMNLP*, 2020.